빈 출발점에서 RAG 아키텍처를 설계한 과정

작성일: 2026-10-14 · Claude 최초 기획을 바탕으로 설계 의도와 후속 선택을 구별한 문서

1. 무엇을 만들려고 했나

최초 목표는 공공 입찰 제안요청서(RFP) 100건에서 근거를 찾아 한국어 질문에 답하는 시스템이었다. 한 문서의 세부 조건, 여러 문서의 비교, 후속 질문, 문서에 없는 질문의 거절을 처리하고, 평가 방법도 직접 만들어 성능을 설명해야 했다. 사용자는 먼저 혼자 끝까지 구현한 뒤 팀원과 방향을 맞추려 했다. [D0]

2026-09-30 Claude 계획에서 확인되는 중심 결정은 **기획과 개발 중 평가는 Claude를 사용하되, 당시 완성 시스템은 로컬 공개 모델로 실행한다**는 것이었다. 작은 모델이 많은 일을 한 번에 처리하도록 두지 않고, 규칙·검색·사전 요약·질문별 처리·검증으로 부담을 나눴다. API 방식은 나중에 공급자 설정으로 바꿀 수 있도록 경계를 만들었다. [D0]

이 문서는 "최종 구조가 이렇게 생겼다"보다, 어떤 문제를 예상했고 무엇을 먼저 만들었으며 어떤 비교로 선택을 좁혔는지를 설명한다. 최초 계획은 의도에 대한 근거이고, 후속 결과 파일은 선택에 대한 근거다. 계획에 적힌 모든 기능이 최종 기본값에 들어간 것은 아니다.

2. 베이스라인은 두 가지 의미로 쓰였다

첫 번째는 프로젝트 초기의 00_baseline이다. 문서를 읽고 고정 길이로 자른 뒤 dense 검색의 상위 근거를 모델에 주고 한 번 답하게 하는 가장 단순한 실험 출발점이다. 이후 개선이 실제 효과가 있는지 비교하는 기준으로 사용했다. [D0, D2]

두 번째는 최근 팀 공동작업용으로 새로 만든 최소 모듈러 기준선이다. 공통 자료형과 빈 공정 폴더에서 시작해 문서 읽기→청킹→벡터화→검색→생성의 연결을 구현했다. 이 기준선은 기존 실험의 00_baseline과 코드·모델·데이터 검증 범위가 다르다. 이름이 비슷하다고 같은 실험으로 취급하면 설계 경위가 잘못 전달된다. [D7]

3. 처음에 확인한 조건이 설계에 미친 영향

원본은 HWP가 대부분이었고 CSV 본문은 불완전했다

자료는 HWP 96개, PDF 4개와 메타데이터 CSV였다. 최초 계획은 CSV의 텍스트가 문서당 89~18,335자, 중간값 2,716자로 원문 전체가 아니라는 점을 기록했다. 따라서 CSV만 검색하면 세부 요구사항이 빠질 수 있어 HWP·PDF 원본을 직접 읽는 단계부터 설계했다. CSV는 기관명·사업명·예산·마감일 등 구조화된 정보를 연결하고, 원본을 읽지 못할 때 대체 본문을 제공하는 역할로 나눴다. [D0]

작은 로컬 모델과 자원 한도가 있었다

최초 기록의 장비는 RTX 5080 16GB와 RAM 31GB였다. 후속 실행에서는 프로젝트 프로세스의 메모리 15GB·CPU 8코어 제한이 설계와 모델 선택에 영향을 줬다. 이 숫자는 당시 기록이며 현재 장비를 새로 측정한 값은 아니다. 모델을 크게 하는 대신 필요한 근거를 좁히고 여러 문서 처리를 나누는 방향을 택했다. [D0, D2]

검색과 생성의 원인을 구별해야 했다

답이 틀렸을 때 문서를 못 찾은 것인지, 근거는 받았지만 잘못 읽은 것인지 알아야 했다. 따라서 검색만 먼저 평가하고, 다음에 답변 파이프라인을 평가했다. ref_claude에는 로컬 모델과 같은 검색 근거를 제공해 주로 생성 단계의 차이를 보도록 했다. 이는 Claude를 독립된 정답 검색기로 사용한 설계가 아니다. [D0, D2]

4. 구현을 어떤 순서로 계획했나

단계	먼저 해결하려던 문제	계획한 작업	뒤 단계와의 관계
P1 환경·읽기 시험	데이터와 모델을 실제로 읽고 실행할 수 있는가	환경 준비, 원본 수 확인, HWP 표본 읽기, 실행 장비 확인	모델 선정 전에 실행 가능성을 확인한다.
P2 관찰 화면	어떤 단계가 진행 중인지 알 수 있는가	진행 대시보드와 호출 기록	품질 점수와 작업 진행 상태를 구분한다.
P3 문서·청크	원문 요구사항을 검색 가능한 형태로 만들 수 있는가	HWP/PDF 추출, 표 글 보존, fixed·section 청크, 기관·사업 메타데이터	임베딩 전에 내용과 문서 ID를 준비한다.
P4 평가 준비	무엇을 잘했다고 판단할 것인가	질문 40개, 정답 문서·핵심 사실, 개발/확인 분리, 채점기	검색·생성 선택 전에 비교 기준을 준비한다.
P5 검색 선택	필요한 문서와 사실을 충분히 가져오는가	임베딩·청킹·검색·재순위를 한 번에 하나씩 비교	불필요한 생성 호출을 줄이고 검색 원인을 분리한다.
P6 생성 선택	작은 모델이 근거를 제대로 사용하게 할 수 있는가	질문 분류, 대화 상태, 문서별 처리, JSON 응답, 검증, 사전 요약	검색 결과가 좋더라도 생성의 회귀를 별도로 확인한다.
P7 사용 화면	선택한 흐름을 사용자가 실행할 수 있는가	질문·설정 화면	화면 선택과 품질 선정은 별개다.
P8 문서·검토	선택 이유와 재현 방법을 설명할 수 있는가	기술 문서, 보고서, 코드 검토	미채택 방법과 한계도 남긴다.

P3와 P4는 병행할 수 있도록 계획했다. 무거운 모델 실행은 순서를 조정했고, 요약 생성과 속도 측정을 동시에 하지 않도록 했다. 처음부터 평가를 마지막 장식으로 붙인 구조는 아니었다. 다만 원문이 얼마나 잘 읽혔는지에 대한 자세한 EDA와 전처리 기준 설계는 늦게 보강됐다. [D0, D1]

5. 라이브러리와 모델은 어떤 기준으로 선택했나

5.1 HWP: olefile

HWP에는 요구사항이 표 안에 있었다. 최초 계획은 OLE 컨테이너의 본문 레코드를 직접 읽고, 후속 구현은 표의 칸을 | 로 연결해 텍스트로 남겼다. 머리말·꼬리말 같은 반복 부분을 제외하고, 잠금 여부를 확인해 필요하면 CSV 본문으로 대체하는 흐름을 뒀다. [D0, D2]

선택에 남아 있는 실측 근거는 속도다. hwp5html은 큰 파일 하나에 46~129초가 걸렸다는 기록이 있고, olefile은 HWP 한 건의 중앙값 0.02초, 최대 0.04초, 전체 약 1.7초로 기록됐다. 따라서 빠르게 본문을 추출하는 방식을 택했다. 그러나 표를 텍스트로 이어 붙이는 것은 원래 표의 행·열 구조를 완전하게 복원하는 것과 다르다. "빠르고 완벽한 HWP 파서"로 선정한 것은 아니다. [D2]

5.2 PDF: PyMuPDF

PDF는 PyMuPDF로 텍스트와 읽기 상태를 확인했다. 당시 PDF 4건의 추출 시간은 합계 약 1.0초로 기록됐다. PDF를 문서 객체와 텍스트로 연결하는 구현은 확인된다. 다만 여러 PDF 라이브러리를 같은 조건으로 비교해 PyMuPDF가 가장 정확하다고 선정한 기록은 이번에 확인하지 못했다. 사용 이유와 경쟁 도구 대비 우승 근거를 구별해야 한다. [D0, D2]

5.3 한국어 낱말 처리: kiwipiepy

Kiwi는 문서 파일을 여는 도구가 아니라 한국어 문장을 낱말로 나누는 도구다. 메인의 BM25에서는 명사·동사·외국어·숫자 등 선택한 품사의 형태를 토큰으로 사용한다. 한국어 검색에서 단순 공백 분리만 쓰지 않도록 설계한 것이다. 이것은 코드와 기획에 나타난 기능적 이유다. 다른 형태소 분석기와의 정확도·속도 비교로 Kiwi를 최종 우승 도구로 선정한 기록은 없다. [D0, D4]

팀원 후보의 Kiwi 변경이 새 개선으로 바로 채택되지 않은 이유도 메인에 이미 같은 계열 기능이 있었고, 후속 native 비교에서 그 후보의 사업명 오류가 해소되지 않았기 때문이다. [D6]

5.4 구성 연결: LangChain, 저장: Chroma

최초 계획은 설정 이름으로 검색기·임베딩·생성기를 조립할 수 있도록 LangChain 부품을 사용했다. providers.py와 retrieve.py에 공급자와 검색 방식 선택을 모아, 실험 파일은 기본값에서 바꿀 줄만 기록하게 했다. 목적은 부품 교체와 원인 비교였다. [D0]

Chroma는 청크·메타데이터·벡터를 로컬에 보관하고 다시 검색하는 역할로 정했다. 임베딩 모델이나 청크 방식이 달라지면 색인을 분리해 다른 벡터 공간을 섞지 않았다. "Chroma가 모든 DB 중 가장 빠르다"는 비교 결과나 "LangChain 없이는 이 기능을 구현할 수 없다"는 선정 근거는 확인되지 않았다. 최초 계획에서 기능을 연결하기 위해 선택한 도구로 설명하는 것이 타당하다.

5.5 임베딩: KURE-v1과 BGE-M3를 비교

KURE는 한국어에 맞춘 후보, BGE-M3는 여러 언어를 다루는 후보로 계획했다. 먼저 후보군을 정하고 같은 개발 30문항의 검색 결과를 비교했다. 전처리 전 기록은 다음과 같다. [D0, D3]

임베딩	hit@1	hit@5	검색 사실 포함률	선택 해석
KURE-v1	0.867	0.967	0.731	상위 5개 회수와 핵심 사실 포함이 더 높아 채택
BGE-M3	0.900	0.933	0.636	첫 정답 순위는 더 좋았지만 선택 우선 지표에서 뒤짐

검색 사다리 코드의 선택 순서는 hit@5 → fact_recall → MRR → recall@5였다. 따라서 "한국어 모델이니까 무조건 선택했다"는 설명보다, 한국어 후보로 넣었고 실제 비교의 우선 지표에서 앞섰다고 설명해야 한다. 이 결과는 해당 데이터·청크·질문에서의 관측이며 다른 도메인까지 KURE의 우위를 입증하지 않는다. [D4]

5.6 생성 모델: EXAONE과 Qwen, 실행되지 않은 Gemma

같은 최종 설정에서 비교한 전처리 전 개발 30문항·2판 채점 기록은 다음과 같다. [D2]

모델	정확성	의도	근거 충실도	거절 정확도	p95
EXAONE 3.5 7.8B	3.83	3.97	4.23	1.000	9.6초
Qwen 3.5 9B	3.77	3.83	4.73	0.857	14.0초
Gemma 4 12B	미측정	미측정	미측정	미측정	미측정

EXAONE은 정확성이 비슷한 상황에서 거절과 속도를 보고 선택했다. Qwen의 근거 충실도는 더 높았으므로 EXAONE이 모든 지표에서 이겼다고 설명해서는 안 된다. Gemma는 당시 평가 환경의 메모리 상한에서 실행하지 못해 제외했다. 이는 낮은 품질 점수로 탈락한 결과가 아니다. [D2]

6. 왜 이런 아키텍처로 나눴나

문서 준비와 질문 처리를 나눴다

원본 파싱·청크·임베딩·색인·문서 요약은 문서를 준비할 때 수행한다. 질문을 받을 때는 대화 맥락→질문 분류→근거 검색→답 생성→검증을 수행한다. 반복 질문마다 같은 문서를 다시 읽거나 같은 요약을 만들지 않도록 경계를 나눴다. 이 구분은 최초 계획의 구현 방식이다. 사전 처리 비용과 캐시 갱신 책임은 별도로 남는다. [D0]

문서 준비 흐름: **HWP·PDF·CSV → 추출·정리 → 청크·메타데이터 → 임베딩·색인 / 문서 요약**

질문 처리 흐름: **질문·대화 상태 → 기관·사업 확인 및 분류 → 유형별 근거 → JSON 답변 → 숫자·날짜 검증 → 답·근거**

규칙으로 확정할 수 있는 일은 코드에 맡겼다

기관·사업명은 CSV 목록과 문자열 유사도로 찾고, 예산·마감일처럼 구조화된 값은 CSV에서 꺼내도록 계획했다. 작은 모델이 이미 확정된 숫자를 다시 추측하게 하지 않으려는 설계였다. 다만 코드의 값 존재 검사만으로 숫자 사이의 관계나 비교 방향까지 보장할 수는 없다. 후속 평가에서 기간 관계와 수량 구성의 문제가 남은 것도 이 한계와 연결된다. [D0, D6]

질문 종류별로 일을 나눴다

한 문서 조회, 여러 문서 비교, 후속 질문, 목록·순위 질문은 필요한 근거와 답변 형태가 다르다. 따라서 route 단계에서 종류를 나누고, 여러 문서는 각각 내용을 뽑은 뒤 합치도록 했다. 실제 기록에서는 작은 모델이 비교 표의 칸을 엇갈리게 채우는 문제가 있어 문서별 답을 이어 붙이고 비교 요약을 쓰도록 바꿨다. [D1, D2]

답변과 근거를 함께 추적했다

JSON의 answer, found, evidence_ids로 답과 근거를 연결하고, 숫자·날짜가 인용 근거에 있는지 확인하도록 했다. 이는 답변 추적과 일부 오류 차단을 위한 설계다. 근거 ID 형식 검사, 숫자 존재 검사, 의미가 정확한지 평가하는 일은 서로 다른 수준이다. 최초 계획의 "근거 없는 문장을 걸러 낸다"는 넓은 목표가 현재 완전한 의미 검증으로 구현됐다고 볼 수는 없다. [D0, D5]

모델 공급자를 경계 뒤에 뒀다

모듈이 로컬 모델의 고유 호출 방식을 곳곳에 직접 쓰지 않도록 공급자 선택을 모았다. 로컬·API 비교에서 파이프라인을 통째로 다시 작성하지 않으려는 목적이었다. 임베딩 공급자가 바뀌면 색인도 다시 만들어야 하므로 생성 모델 교체와 검색 벡터 교체를 구별했다. [D0]

7. 처음 계획과 실제 선택은 어떻게 달라졌나

초기에는 section 청크, 하이브리드, MMR, Multi-Query, 재순위를 모두 후보로 넣었다. 하지만 검색 사다리를 실행한 뒤 기본값보다 불리한 후보는 생성 사다리에서 제외했다. 복잡한 방법을 구현했다고 무조건 켜는 구조가 아니었다. [D0, D1, D2]

변화	초기 의도	실제 비교 뒤의 선택
section 청크	RFP의 제목과 요구사항 경계를 살린다	검색 사실 포함률 0.526으로 fixed 0.731보다 낮아 기본에서 제외
하이브리드	의미 검색과 낱말 검색을 결합한다	hit@5는 같았지만 첫 정답·MRR·사실 포함률에서 불리해 기본 dense 유지
재순위	후보 중 더 관련 있는 근거를 앞으로 옮긴다	후속 기준도 통과하지 못해 꺼 둠
Top-10	더 많은 사실을 모델에 제공한다	검색 사실 포함률은 올랐지만 답변 정확성 증가 0.03점, p95 증가 3.5초로 Top-5 유지
질문 분류	질문별로 필요한 처리를 나눈다	2판 정확성 3.07→3.83, 과도한 거절 0.174→0.043으로 가장 큰 관측 개선

변화	초기 의도	실제 비교 뒤의 선택
검증·사전 요약	답의 오류를 줄이고 넓은 질문에 대응한다	함께 추가한 07_verify에서 과도한 거절 0까지 내려갔지만, 둘의 개별 효과는 실험하지 않음

07_verify는 한 요소만 더한 순수 실험이 아니라 검증과 사전 요약을 함께 추가했다. 따라서 그 결과를 검증 하나의 효과로 설명하지 않는다. 한 번에 하나씩 바꾸는 사다리 방식도 모든 조합을 시험한 것이 아니므로 전역 최적성을 입증하지 않는다.

8. 설계 당시 부족했던 부분과 뒤의 보강

원문 검수와 전처리 설계는 나중에 보강됐다

2026-10-02 사용자는 문서 검수, 모델이 읽을 수 있는지 판별, 목차·메타데이터 조사와 전처리 설계가 부족하다고 지적했다. 이후 EDA로 글자 수·한글·깨진 글자·표·그림을 조사하고, 전처리 뒤 청크·색인·요약을 다시 만들도록 했다. 이 순서는 처음부터 완벽히 설계한 절차가 아니라 실제 부족한 점을 확인하고 보강한 과정이다. [D1, D8]

읽힘 판정에는 2,000자 이상, 한글 비율 0.3 이상, 깨진 글자 1% 미만, 글 없는 PDF 쪽 20% 이하 등이 쓰였다. 기술문서는 이 숫자와 "문제 문서 5건 또는 전체 글자의 0.5%"라는 전처리 단계 도입 기준이 사전에 정한 임의 값이며 기록된 근거가 없다고 명시한다. 이를 검증된 표준으로 포장해서는 안 된다. [D2]

청크 1,000/200의 최적 크기는 입증하지 않았다

fixed와 section은 비교했지만 fixed의 모든 크기를 탐색한 것은 아니다. 최초 계획에 1,000자·겹침 200자가 출발값으로 적혀 있으며, 왜 정확히 그 숫자인지에 대한 별도 근거는 확인되지 않았다. fixed 방식의 상대적 결과와 1,000/200이라는 수치의 최적성을 구별해야 한다. [D0, D2]

채점 기준을 두 차례 강화했다

초기 채점의 느슨함을 사용자가 지적해 2판에서는 질문 의도와 누락을 더 엄격하게 평가했다. 3판에서는 답을 주장 단위로 나누고 받은 근거 전체와 대조하도록 바꿨다. 1판의 07_verify 4.20/4.40과 2판·3판의 점수를 같은 성적표로 섞을 수 없는 이유다. [D2, D8]

전처리 전후 환각 문항은 처음 3→8로 보였지만, 동일 답 재채점에서 전처리 전 8, 후 7로 달라졌다. 기록은 채점 변동으로도 설명될 수 있어 전처리가 환각을 늘렸거나 줄였다고 결론 내리지 않았다. 온도 0에서 한 번 같은 답이 재현된 관측도 모든 모델·버전의 완전한 결정성을 보장하지 않는다. [D2]

9. 이후 팀원 통합과 최소 모듈러로 이어진 이유

처음의 모듈 경계는 한 사람이 검색·생성 실험을 바꾸기 위한 구조였다. 팀원이 각자 만든 코드가 들어오자 출처 고정, 원형 보존 A, 제한적 개선 B, 같은 메인 비교, 승인 기록이 추가로 필요해졌다. 이를 통해 "누가 만든 알고리즘인지"와 "통합하면서 무엇을 고쳤는지"를 구별했다. 팀원 평가 결과가 완성 구조의 설명에만 묻히지 않게 한 것이다. [D6]

최근 새 모듈러 저장소는 이 흐름을 더 단순한 공통 규약으로 다시 시작했다. Document → Chunk → SearchHit → Answer를 공정 경계로 두고, 팀원이 한 공정씩 바꿔 볼 수 있도록 했다. 현재 기본 Haiku·TF-IDF는 연결이 확인된 경로이며, KURE·EXAONE을 품질 경쟁에서 이긴 구성으로 선택한 것은 아니

다. 분류·기억·의미 검증·평가는 다음 구현 범위로 남겨 두었다. [D7]

따라서 초기 기획의 배경, 기존 07_verify의 선택 이유, 새 최소 모듈러의 기본값은 서로 연결되지만 같은 결정은 아니다. **초기에는 작은 로컬 모델의 부담을 줄이는 구조를 설계했고, 기존 실험에서는 관측된 효과와 속도로 기능을 좁혔으며, 새 기준선에서는 팀 공동 실험을 위해 최소 연결과 교체 규약을 우선했다.**

10. 설계 경위를 설명할 때 사용할 문장

"RFP 원문이 HWP와 표 중심이고 CSV 본문은 불완전했기 때문에, 먼저 원본 추출과 메타데이터 연결을 설계했다. 작은 로컬 모델을 사용할 계획이어서 구조화된 값은 코드로 처리하고, 질문을 종류별로 나눠 필요한 근거만 전달했다. 검색과 생성을 분리해 평가한 뒤 KURE와 dense Top-5를 선택했고, 질문 분류·사전 요약·일부 검증을 더했다. 처음 후보로 넣었던 고급 검색 방법도 실험에서 이점이 확인되지 않으면 기본값에서 제외했다. 원문 검수와 채점의 부족한 점은 후속 검토에서 보강했다. 이 구조는 처음부터 정답으로 정한 완성 설계가 아니라, 실행 조건과 평가 결과를 보며 선택을 좁혀 간 결과다."

11. 원본 근거

Claude 파일은 당시 작성자의 설계 의도를 확인하기 위해 읽었다. 계획의 문장을 현재 구현 완료 상태로 그대로 옮기지 않고, 진행 기록·코드·결과와 대조했다. 이번에는 Claude 대화 로그와 계정 설정을 읽거나 실행하지 않았고 원본 작업 파일도 수정하지 않았다.

ID	읽은 자료	용도
D0		2026-09-30 최초 방향, 자원·데이터 조건, 단계별 계획
D1	진행 기록	실제 구현 순서, 검색·생성 사다리 변경, EDA 보강 요청
D2	기술문서	도구 선택, 비교 결과, 판별 한계, 부족한 선정 근거
D3	KURE 결과, BGE 결과	검색 수치 대조
D4	검색 코드, 색인 코드, 입력 코드	실제 선택 우선순위, 한국어 토큰화·추출 역할
D5	기본 설정, 최종 설정	모델·청크·검색·검증의 저장 상태
D6	팀 버전 관리 검토, 모듈 통합 기록, native 평가	이후 팀원 비교와 실제 반영 경위
D7	최소 모듈러 README, 공정별 기준	기존 최종 구성과 새 기준선의 목적 구분
D8		기존 채점 판 혼합 금지와 전처리 보강 기획

정확한 원본 위치·해시는 같은 폴더의 근거목록.json에 기록했다. 기록에 없는 선택 이유는 추정으로 보충하지 않았다. 이 문서 작성 중 새 모델 평가나 런타임 변경은 수행하지 않았다.